

# Real-Life Application Using OOP in C++

## Title: Library Management System

### 1. Introduction

A **Library Management System** is a real-life application used to manage books, users, and borrowing activities in a library.

This system helps automate tasks like issuing books, returning books, and maintaining records.

### 2. OOP Concepts Used

- **Class & Object** → Represents real-world entities like Book and User
- **Encapsulation** → Data hiding using private variables
- **Inheritance** → Extending base class (User → Student/Teacher)
- **Polymorphism** → Same function behaving differently
- **Abstraction** → Showing only necessary details

### 3. Class Design

#### Class: Book

- Attributes: Book ID, Title, Author, Availability
- Methods: Add book, Display book

#### Class: User

- Attributes: User ID, Name
- Methods: Issue book, Return book

#### Derived Classes

- Student
- Teacher

### 4. C++ Implementation

```
#include<iostream>
#include<vector>
using namespace std;

// Book Class
class Book {
private:
    int id;
    string title;
    bool issued;

public:
    Book(int i, string t) {
        id = i;
        title = t;
        issued = false;
    }

    int getId() { return id; }

    void display() {
        cout << "ID: " << id << " | Title: " << title
            << " | Status: " << (issued ? "Issued" : "Available") << endl;
    }

    bool isAvailable() { return !issued; }

    void issue() { issued = true; }

    void returnBook() { issued = false; }
};

// Library Class
class Library {
private:
    vector<Book> books;

public:
    void addBook(int id, string title) {
        books.push_back(Book(id, title));
        cout << "Book Added Successfully!\n";
    }
}
```

```

void showBooks() {
    for(auto &b : books) {
        b.display();
    }
}

void issueBook(int id) {
    for(auto &b : books) {
        if(b.getId() == id) {
            if(b.isAvailable()) {
                b.issue();
                cout << "Book Issued!\n";
            } else {
                cout << "Already Issued!\n";
            }
        }
        return;
    }
}

cout << "Book Not Found!\n";
}

void returnBook(int id) {
    for(auto &b : books) {
        if(b.getId() == id) {
            b.returnBook();
            cout << "Book Returned!\n";
            return;
        }
    }
}

cout << "Book Not Found!\n";
}
};

```

// Main Function

```

int main() {
    Library lib;
    int choice, id;
    string title;

    do {
        cout << "\n--- Library Menu ---\n";
        cout << "1. Add Book\n";
        cout << "2. Show Books\n";
    }
}

```

```
cout << "3. Issue Book\n";
cout << "4. Return Book\n";
cout << "5. Exit\n";
cout << "Enter choice: ";
cin >> choice;
```

```
switch(choice) {
    case 1:
        cout << "Enter ID and Title: ";
        cin >> id;
        cin.ignore();
        getline(cin, title);
        lib.addBook(id, title);
        break;
```

```
    case 2:
        lib.showBooks();
        break;
```

```
    case 3:
        cout << "Enter Book ID: ";
        cin >> id;
        lib.issueBook(id);
        break;
```

```
    case 4:
        cout << "Enter Book ID: ";
        cin >> id;
        lib.returnBook(id);
        break;
```

```
}
```

```
} while(choice != 5);
```

```
return 0;
```

```
}
```

```
--- Library Menu ---
1. Add Book
2. Show Books
3. Issue Book
4. Return Book
5. Exit
Enter choice: 1
Enter ID and Title: 101 C++ book
Book Added Successfully!

--- Library Menu ---
1. Add Book
2. Show Books
3. Issue Book
4. Return Book
5. Exit
Enter choice: 3
Enter Book ID: 101
Book Issued!
```

## 5. Features of the System

- Add and display books
- Issue and return books
- Different user roles (Student/Teacher)
- Uses real-life OOP concepts

## 6. Conclusion

This project demonstrates how **Object-Oriented Programming** can model real-world systems efficiently.

The system is scalable and can be extended with features like database integration and GUI.